```
/*
================================================================================
=
  PROYECTO: SISTEMA DE GESTIÓN PARA FERRETERÍA CABRERA
  MOTOR:    Microsoft SQL Server (T-SQL)
  MÓDULOS:  1. Inventario | 2. Nómina | 3. Punto de Venta (POS) y Caja

================================================================================
= */

USE master;
GO

-- Si la base de datos ya existe, la elimina para recrearla limpia sin errores
IF DB_ID('FerreteriaCabreraDB') IS NOT NULL
BEGIN
   ALTER DATABASE FerreteriaCabreraDB SET SINGLE_USER WITH ROLLBACK
IMMEDIATE;
   DROP DATABASE FerreteriaCabreraDB;
END
GO

CREATE DATABASE FerreteriaCabreraDB;
GO

USE FerreteriaCabreraDB;
GO

/*
================================================================================
=
  SEGURIDAD Y AUDITORÍA BASE (Para alinear con Diagrama de Clases y Cap 5.4)

================================================================================
= */

CREATE TABLE Usuario (
   id_usuario INT IDENTITY(1,1) PRIMARY KEY,
   nombre_usuario VARCHAR(50) NOT NULL UNIQUE,
   rol VARCHAR(20) NOT NULL DEFAULT 'Encargado',
   clave_hash VARCHAR(255) NOT NULL,
   activo BIT NOT NULL DEFAULT 1
);
GO
```

```
CREATE TABLE BitacoraMovimientos (
    id_bitacora INT IDENTITY(1,1) PRIMARY KEY,
    fecha_hora DATETIME NOT NULL DEFAULT GETDATE(),
    modulo VARCHAR(30) NOT NULL,
    accion VARCHAR(100) NOT NULL,
    detalle VARCHAR(255) NULL
);
GO

/*
================================================================================
   MÓDULO 1: INVENTARIO Y CATÁLOGO (Normalizado en 3FN - Búsqueda por código)
================================================================================ */

CREATE TABLE Categoria (
    id_categoria INT IDENTITY(1,1) PRIMARY KEY,
    nombre_categoria VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(150) NULL
);
GO

CREATE TABLE UnidadMedida (
    id_unidad VARCHAR(5) PRIMARY KEY, -- Ej: 'PZA', 'KG', 'MT', 'LT', 'CAJ'
    nombre_unidad VARCHAR(30) NOT NULL
);
GO

CREATE TABLE Producto (
    codigo_barras VARCHAR(20) PRIMARY KEY, -- Búsqueda directa por código interno o de barras
    nombre VARCHAR(100) NOT NULL,
    id_categoria INT NOT NULL,
    id_unidad VARCHAR(5) NOT NULL,
    precio_compra DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    precio_venta DECIMAL(10,2) NOT NULL,
    stock_actual DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    stock_minimo DECIMAL(10,2) NOT NULL DEFAULT 5.00,
    activo BIT NOT NULL DEFAULT 1,
    CONSTRAINT FK_Producto_Categoria FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria),
```

```
    CONSTRAINT FK_Producto_Unidad FOREIGN KEY (id_unidad) REFERENCES
UnidadMedida(id_unidad),
    CONSTRAINT CHK_Precios_Positivos CHECK (precio_venta >= 0 AND precio_compra >=
0),
    CONSTRAINT CHK_Stock_NoNegativo CHECK (stock_actual >= 0) -- Evita inventario
negativo
);
GO

-- Índice para acelerar las búsquedas por nombre en mostrador
CREATE NONCLUSTERED INDEX IX_Producto_Nombre ON Producto(nombre);
GO

/*
================================================================================
   MÓDULO 2: NÓMINA INFORMAL (Control de Empleados, Niveles, Deptos y Raya)

================================================================================ */

-- 1. Tabla Niveles (Relacionada con Empleados)
CREATE TABLE Niveles (
    nivel VARCHAR(3) PRIMARY KEY,          -- Ej: 'N01', 'N02', 'N03'
    descripcion VARCHAR(50) NOT NULL,      -- Ej: 'Ayudante General', 'Mostrador', 'Bodeguero'
    saldo_semanal DECIMAL(10,2) NOT NULL,  -- Lo que se le paga a la semana (Raya base)
    CONSTRAINT CHK_Saldo_Positivo CHECK (saldo_semanal > 0)
);
GO

-- 2. Tabla Departamento (Relacionada con Empleados)
CREATE TABLE Departamento (
    depto VARCHAR(3) PRIMARY KEY,          -- Ej: 'D01', 'D02', 'D03'
    nombre_dpto VARCHAR(20) NOT NULL,      -- Ej: 'Mostrador', 'Bodega', 'Patio/Entregas'
    ubicacion VARCHAR(5) NOT NULL          -- Ej: 'LOC01', 'BOD01', 'PAT01'
);
GO

-- 3. Tabla Empleados (Con todos los campos solicitados)
CREATE TABLE Empleados (
    clave_empleado INT IDENTITY(1,1) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    nivel VARCHAR(3) NOT NULL,
    depto VARCHAR(3) NOT NULL,
```

```
    curp VARCHAR(18) NOT NULL UNIQUE,
    nombre_esposa VARCHAR(100) NULL,       -- Opcional por si es soltero
    telefono_esposa VARCHAR(15) NULL,
    telefono_emergencia VARCHAR(15) NOT NULL,
    direccion_empleado VARCHAR(150) NOT NULL,
    fecha_ingreso DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
    activo BIT NOT NULL DEFAULT 1,
    CONSTRAINT FK_Empleados_Niveles FOREIGN KEY (nivel) REFERENCES Niveles(nivel),
    CONSTRAINT FK_Empleados_Departamento FOREIGN KEY (depto) REFERENCES
Departamento(depto)
);
GO

/*
================================================================================
  MÓDULO 3: PUNTO DE VENTA (POS), CORTE DE CAJA Y CONEXIÓN CON NÓMINA

================================================================================ */

-- Control de apertura y cierre de caja diario
CREATE TABLE CorteCaja (
    id_corte INT IDENTITY(1,1) PRIMARY KEY,
    id_usuario INT NOT NULL,
    fecha_apertura DATETIME NOT NULL DEFAULT GETDATE(),
    fecha_cierre DATETIME NULL,
    fondo_inicial DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    total_ingresos_ventas DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    total_egresos_nomina DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    saldo_final_esperado AS (fondo_inicial + total_ingresos_ventas - total_egresos_nomina),
    estado VARCHAR(15) NOT NULL DEFAULT 'ABIERTO', -- 'ABIERTO' o 'CERRADO'
    CONSTRAINT FK_CorteCaja_Usuario FOREIGN KEY (id_usuario) REFERENCES
Usuario(id_usuario)
);
GO

-- Tabla de Pagos de Nómina (Vincula al Empleado con el dinero que sale de la Caja)
CREATE TABLE PagosNomina (
    id_pago INT IDENTITY(1,1) PRIMARY KEY,
    clave_empleado INT NOT NULL,
    id_corte INT NOT NULL,                 -- Caja de donde se tomó el efectivo para pagar la raya
    fecha_pago DATETIME NOT NULL DEFAULT GETDATE(),
    dias_trabajados INT NOT NULL DEFAULT 6,
```

```
    sueldo_base_semanal DECIMAL(10,2) NOT NULL,
    adelantos_descuentos DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    total_pagado DECIMAL(10,2) NOT NULL,
    observaciones VARCHAR(100) NULL,
    CONSTRAINT FK_PagosNomina_Empleado FOREIGN KEY (clave_empleado)
REFERENCES Empleados(clave_empleado),
    CONSTRAINT FK_PagosNomina_CorteCaja FOREIGN KEY (id_corte) REFERENCES
CorteCaja(id_corte),
    CONSTRAINT CHK_TotalPagado CHECK (total_pagado >= 0)
);
GO

-- Tabla Venta (Alineada a tu Diagrama de Clases)
CREATE TABLE Venta (
    id_venta INT IDENTITY(1,1) PRIMARY KEY, -- Folio de la nota
    id_usuario INT NOT NULL,
    id_corte INT NOT NULL,
    fecha_hora DATETIME NOT NULL DEFAULT GETDATE(),
    total_cobrado DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    efectivo_pagado DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    cambio DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    CONSTRAINT FK_Venta_Usuario FOREIGN KEY (id_usuario) REFERENCES
Usuario(id_usuario),
    CONSTRAINT FK_Venta_CorteCaja FOREIGN KEY (id_corte) REFERENCES
CorteCaja(id_corte)
);
GO

-- Tabla DetalleVenta (Alineada a tu Diagrama de Clases)
CREATE TABLE DetalleVenta (
    id_detalle INT IDENTITY(1,1) PRIMARY KEY,
    id_venta INT NOT NULL,
    codigo_barras VARCHAR(20) NOT NULL,
    cantidad DECIMAL(10,2) NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(10,2) NOT NULL,
    CONSTRAINT FK_DetalleVenta_Venta FOREIGN KEY (id_venta) REFERENCES
Venta(id_venta),
    CONSTRAINT FK_DetalleVenta_Producto FOREIGN KEY (codigo_barras) REFERENCES
Producto(codigo_barras),
    CONSTRAINT CHK_Cantidad_Positiva CHECK (cantidad > 0)
);
GO
```

```
/*
================================================================================
=
  VISTAS (Reportes rápidos de Inventario y Nómina)

================================================================================
= */

-- 1. Vista de Alerta de Stock Mínimo (Requisito RQ-02 de la Matriz de Trazabilidad)
CREATE VIEW vw_AlertaStockMinimo AS
SELECT
   p.codigo_barras,
   p.nombre AS producto,
   c.nombre_categoria AS categoria,
   p.stock_actual,
   p.stock_minimo,
   u.nombre_unidad AS unidad,
   p.precio_venta
FROM Producto p
INNER JOIN Categoria c ON p.id_categoria = c.id_categoria
INNER JOIN UnidadMedida u ON p.id_unidad = u.id_unidad
WHERE p.stock_actual <= p.stock_minimo AND p.activo = 1;
GO

-- 2. Vista de Padrón de Empleados con su Sueldo Semanal y Departamento
CREATE VIEW vw_PadronEmpleados AS
SELECT
   e.clave_empleado,
   e.nombre AS nombre_empleado,
   e.curp,
   d.depto AS clave_depto,
   d.nombre_dpto,
   d.ubicacion,
   n.nivel AS clave_nivel,
   n.descripcion AS puesto,
   n.saldo_semanal,
   e.nombre_esposa,
   e.telefono_esposa,
   e.telefono_emergencia,
   e.fecha_ingreso
FROM Empleados e
INNER JOIN Niveles n ON e.nivel = n.nivel
INNER JOIN Departamento d ON e.depto = d.depto
WHERE e.activo = 1;
```

```
GO

/*
================================================================================
  DISPARADORES (TRIGGERS) PARA AUTOMATIZACIÓN Y AUDITORÍA
================================================================================ */

-- Trigger 1: Descuenta el stock automáticamente y actualiza la caja al insertar un DetalleVenta
CREATE TRIGGER trg_DescontarStockYActualizarVenta
ON DetalleVenta
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Descontar el stock en la tabla Producto
    UPDATE p
    SET p.stock_actual = p.stock_actual - i.cantidad
    FROM Producto p
    INNER JOIN inserted i ON p.codigo_barras = i.codigo_barras;

    -- 2. Registrar en bitácora si algún producto llegó a su stock mínimo tras la venta
    INSERT INTO BitacoraMovimientos (modulo, accion, detalle)
    SELECT
        'INVENTARIO',
        'ALERTA STOCK MÍNIMO',
        'El código ' + p.codigo_barras + ' (' + p.nombre + ') bajó a ' + CAST(p.stock_actual AS
VARCHAR(20))
    FROM Producto p
    INNER JOIN inserted i ON p.codigo_barras = i.codigo_barras
    WHERE p.stock_actual <= p.stock_minimo;
END;
GO

-- Trigger 2: Cuando se registra un pago de nómina, lo descuenta automáticamente del Corte
de Caja activo
CREATE TRIGGER trg_DescontarNominaDeCaja
ON PagosNomina
AFTER INSERT
AS
BEGIN
```

```
    SET NOCOUNT ON;

    -- Sumar al total de egresos de nómina en el corte de caja correspondiente
    UPDATE c
    SET c.total_egresos_nomina = c.total_egresos_nomina + i.total_pagado
    FROM CorteCaja c
    INNER JOIN inserted i ON c.id_corte = i.id_corte;

    -- Dejar rastro en la bitácora de auditoría
    INSERT INTO BitacoraMovimientos (modulo, accion, detalle)
    SELECT
        'NÓMINA / CAJA',
        'PAGO DE RAYA SEMANAL',
        'Pago a empleado clave ' + CAST(i.clave_empleado AS VARCHAR(10)) + ' por $' +
CAST(i.total_pagado AS VARCHAR(20))
    FROM inserted i;
END;
GO

/*
=========================================================================
=
  PROCEDIMIENTOS ALMACENADOS (Con Bloqueo Pesimista y ACID - Cap 5.4)

=========================================================================
= */

-- 1. Procedimiento para Pagar la Raya Semanal a un Empleado
CREATE PROCEDURE sp_PagarRayaEmpleado
    @clave_empleado INT,
    @id_corte INT,
    @dias_trabajados INT = 6, -- Semana laboral estándar de 6 días
    @adelantos_descuentos DECIMAL(10,2) = 0.00,
    @observaciones VARCHAR(100) = 'Pago de raya semanal completa'
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @saldo_semanal DECIMAL(10,2);
        DECLARE @pago_proporcional DECIMAL(10,2);
        DECLARE @total_pagar DECIMAL(10,2);
```

```
        -- Obtener el saldo semanal según el nivel del empleado
        SELECT @saldo_semanal = n.saldo_semanal
        FROM Empleados e
        INNER JOIN Niveles n ON e.nivel = n.nivel
        WHERE e.clave_empleado = @clave_empleado AND e.activo = 1;

        IF @saldo_semanal IS NULL
        BEGIN
            RAISERROR('El empleado no existe o está inactivo.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        -- Si trabajó los 6 días se le paga su saldo_semanal íntegro; si faltó, se calcula
proporcional
        SET @pago_proporcional = ROUND((@saldo_semanal / 6.0) * @dias_trabajados, 2);
        SET @total_pagar = @pago_proporcional - @adelantos_descuentos;

        IF @total_pagar < 0 SET @total_pagar = 0;

        -- Insertar el pago (El Trigger descontará automáticamente el efectivo del CorteCaja)
        INSERT INTO PagosNomina (clave_empleado, id_corte, dias_trabajados,
sueldo_base_semanal, adelantos_descuentos, total_pagado, observaciones)
        VALUES (@clave_empleado, @id_corte, @dias_trabajados, @saldo_semanal,
@adelantos_descuentos, @total_pagar, @observaciones);

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        DECLARE @ErrorMsg NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMsg, 16, 1);
    END CATCH
END;
GO

-- 2. Procedimiento Transaccional de Venta (Con bloqueo pesimista UPDLOCK según Cap 5.4)
CREATE PROCEDURE sp_ProcesarVentaUnitaria
    @id_usuario INT,
    @id_corte INT,
    @codigo_barras VARCHAR(20),
    @cantidad DECIMAL(10,2),
    @efectivo_recibido DECIMAL(10,2)
AS
```

```
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @stock_disponible DECIMAL(10,2);
        DECLARE @precio DECIMAL(10,2);
        DECLARE @subtotal DECIMAL(10,2);
        DECLARE @cambio DECIMAL(10,2);
        DECLARE @nuevo_id_venta INT;

        -- Bloqueo pesimista (UPDLOCK, HOLDLOCK) para evitar concurrencia al verificar stock
        SELECT
            @stock_disponible = stock_actual,
            @precio = precio_venta
        FROM Producto WITH (UPDLOCK, HOLDLOCK)
        WHERE codigo_barras = @codigo_barras AND activo = 1;

        IF @stock_disponible IS NULL
        BEGIN
            RAISERROR('El código del producto no existe.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        IF @stock_disponible < @cantidad
        BEGIN
            RAISERROR('Stock insuficiente en bodega para cubrir la venta.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        SET @subtotal = ROUND(@cantidad * @precio, 2);

        IF @efectivo_recibido < @subtotal
        BEGIN
            RAISERROR('El efectivo recibido es menor al total a cobrar.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        SET @cambio = @efectivo_recibido - @subtotal;

        -- 1. Crear el encabezado de la venta
```

```
        INSERT INTO Venta (id_usuario, id_corte, total_cobrado, efectivo_pagado, cambio)
        VALUES (@id_usuario, @id_corte, @subtotal, @efectivo_recibido, @cambio);

        SET @nuevo_id_venta = SCOPE_IDENTITY();

        -- 2. Insertar el detalle (El Trigger trg_DescontarStockYActualizarVenta descontará el
stock)
        INSERT INTO DetalleVenta (id_venta, codigo_barras, cantidad, precio_unitario, subtotal)
        VALUES (@nuevo_id_venta, @codigo_barras, @cantidad, @precio, @subtotal);

        -- 3. Sumar el ingreso al Corte de Caja activo
        UPDATE CorteCaja
        SET total_ingresos_ventas = total_ingresos_ventas + @subtotal
        WHERE id_corte = @id_corte;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
END;
GO

/*
================================================================================
  DATOS DE PRUEBA (Para que pruebes el sistema de inmediato y veas que funciona)
================================================================================ */

-- 1. Usuario encargado del sistema
INSERT INTO Usuario (nombre_usuario, rol, clave_hash)
VALUES ('admin_cabrera', 'Encargado',
'8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918');

-- 2. Categorías y Unidades de Medida (Módulo Inventario)
INSERT INTO Categoria (nombre_categoria, descripcion) VALUES
('Herramienta Manual', 'Martillos, desarmadores, pinzas y llaves'),
('Plomería y Tubería', 'Tubos PVC, conexiones, llaves de paso y pegamentos'),
('Material Eléctrico', 'Cables, apagadores, contactos y cinta aislante'),
('Fijación y Tornillería', 'Clavos, pijas, tornillos y taquetes');
```

```
INSERT INTO UnidadMedida (id_unidad, nombre_unidad) VALUES
('PZA', 'Pieza'),
('KG', 'Kilogramo'),
('MT', 'Metro'),
('CAJ', 'Caja');

-- 3. Productos ferreteros buscados por código
INSERT INTO Producto (codigo_barras, nombre, id_categoria, id_unidad, precio_compra,
precio_venta, stock_actual, stock_minimo) VALUES
('FER-001', 'Martillo uña curva 16 oz Truper', 1, 'PZA', 110.00, 165.00, 15, 5),
('FER-002', 'Cinta Teflón 3/4 rollo 10m', 2, 'PZA', 8.00, 18.00, 4, 10), -- Ya en alerta de stock
mínimo
('FER-003', 'Cable THW Calibre 12 Indiana', 3, 'MT', 14.50, 22.00, 100, 20),
('FER-004', 'Clavo con cabeza 2.5 pulgadas', 4, 'KG', 35.00, 55.00, 30, 8);

-- 4. Niveles y Departamentos (Módulo Nómina)
INSERT INTO Niveles (nivel, descripcion, saldo_semanal) VALUES
('N01', 'Encargado de Mostrador', 2400.00),
('N02', 'Bodeguero y Despachador', 1800.00),
('N03', 'Ayudante General / Patio', 1500.00);

INSERT INTO Departamento (depto, nombre_dpto, ubicacion) VALUES
('D01', 'Mostrador Ventas', 'LOC01'),
('D02', 'Almacén General', 'BOD01'),
('D03', 'Patio y Materiales', 'PAT01');

-- 5. Empleados con todos los datos solicitados
INSERT INTO Empleados (nombre, nivel, depto, curp, nombre_esposa, telefono_esposa,
telefono_emergencia, direccion_empleado, fecha_ingreso) VALUES
('Roberto Hernández Gómez', 'N01', 'D01', 'HEGR880512HVZRRN01', 'Laura Méndez Ruiz',
'8461002233', '8461009988', 'Calle Hidalgo #204, Col. Centro, Pánuco', '2023-02-15'),
('Carlos Alberto Pérez Loredo', 'N02', 'D02', 'PELC950823HVZPLR02', 'Sofía Castillo Cruz',
'8461123344', '8332154477', 'Av. Carranza #112, Col. Revolución, Pánuco', '2024-06-10'),
('Luis Fernando Torres Díaz', 'N03', 'D03', 'TODL011104HVZTRL05', NULL, NULL,
'8461056677', 'C. Benito Juárez S/N, Moralillo', '2025-01-20');

-- 6. Abrir un Corte de Caja con $1,500 de fondo inicial
INSERT INTO CorteCaja (id_usuario, fondo_inicial) VALUES (1, 1500.00);
GO

-- 7. Simular 2 Ventas por código usando el Procedimiento Almacenado
EXEC sp_ProcesarVentaUnitaria @id_usuario = 1, @id_corte = 1, @codigo_barras =
'FER-001', @cantidad = 2, @efectivo_recibido = 500.00;
```

```
EXEC sp_ProcesarVentaUnitaria @id_usuario = 1, @id_corte = 1, @codigo_barras =
'FER-004', @cantidad = 3, @efectivo_recibido = 200.00;
GO

-- 8. Simular el pago de la raya semanal del empleado 2 (Carlos) descontándole $100 de un
préstamo/adelanto
EXEC sp_PagarRayaEmpleado @clave_empleado = 2, @id_corte = 1, @dias_trabajados = 6,
@adelantos_descuentos = 100.00, @observaciones = 'Se descontaron $100 de adelanto del
miércoles';
GO

/*
==========================================================================
=
   CONSULTAS DE VERIFICACIÓN (Se ejecutan al final para mostrar resultados)

==========================================================================
= */
SELECT * FROM vw_PadronEmpleados;
SELECT * FROM vw_AlertaStockMinimo;
SELECT * FROM CorteCaja;
SELECT * FROM BitacoraMovimientos;
GO
```